Work Distribution, Design Patterns, and Design Principles

Project: UPI Payment Gateway Simulator

---

1. Architectural Overview

The system follows the **MVC (Model–View–Controller) Architecture Pattern**, enforced by the Spring Boot framework.

- **Model Layer**: Domain entities such as Merchant, Transaction, Refund, Settlement, and CallbackLog.
- **View Layer**: React-based web user interface for merchants and administrators.
- **Controller Layer**: REST controllers exposing APIs for payment operations.
- **Service Layer**: Core business logic.
- **Repository Layer**: Database access using JPA/Hibernate.
- **Database**: Relational database (MySQL/PostgreSQL).

The MVC architecture ensures separation of concerns, maintainability, and scalability.

---

2. Design Patterns Used (4 Total)

As per guidelines, the project uses **four design patterns**, including Creational, Behavioral, and the framework-enforced architectural pattern.

Design Pattern	Type	Responsibility
MVC	Architectural (Framework-enforced)	Overall system structure
Factory Pattern	Creational	Creation of payment processors
Strategy Pattern	Behavioral	Handling multiple payment methods

Observer Pattern	Behavioral	Merchant callback notifications
Singleton Pattern	Creational	Centralized configuration management

---

3. Design Principles Used (4 Total)

Each team member is responsible for applying **one design principle** in their owned use case.

Design Principle	Description
SRP – Single Responsibility Principle	Each class has a single responsibility
OCP – Open Closed Principle	System open for extension, closed for modification
DIP – Dependency Inversion Principle	Dependence on abstractions, not concrete classes
LSP / ISP	Substitutability and small, role-specific interfaces

---

4. Team Member–Wise Responsibility Allocation

Each team member owns **one major use case completely**, including:

- UI implementation
- Backend logic
- Database persistence
- UML diagrams related to the use case

4.1 Gopal – Merchant Onboarding & Authentication

Major Use Case

- Merchant Registration

- Merchant Authentication

Responsibilities

- Implement merchant onboarding workflow
- Merchant entity and repository
- Registration and login APIs
- Merchant onboarding UI screens
- UML diagrams: Use Case, Class, Activity

Design Pattern Applied

- Factory Pattern (Creational)

Design Principle Applied

- Single Responsibility Principle (SRP)

4.2 Hemanth – Payment Initiation & Processing

Major Use Case

- Payment Initiation
- Payment Verification

Responsibilities

- Payment request handling
- Transaction creation and processing
- Payment processor selection
- Payment UI flow
- UML diagrams: Activity Diagram for payment flow

Design Pattern Applied

- Strategy Pattern (Behavioral)

Design Principle Applied

- Open Closed Principle (OCP)

4.3 Harshith – Callback & Transaction Status Management

Major Use Case

- Merchant Callback Notification
- Transaction Status Tracking

Responsibilities

- Callback notification mechanism
- Transaction state updates
- Callback logging
- UI for viewing transaction status
- UML diagrams: State Diagram (Transaction Lifecycle)

Design Pattern Applied

- Observer Pattern (Behavioral)

Design Principle Applied

- Dependency Inversion Principle (DIP)

4.4 Kartik – Refund Processing & Settlement Management

Major Use Case

- Refund Processing
- Transaction Settlement and Reports

Responsibilities

- Refund request handling
- Settlement calculation logic
- Settlement reports UI
- Admin-level operations
- UML diagrams: Class and Activity diagrams

Design Pattern Applied

- Singleton Pattern (Creational)

Design Principle Applied

- Liskov Substitution Principle / Interface Segregation Principle

---

5. Minor Use Case Distribution

Team Member	Minor Use Case
Gopal	Merchant Profile Management
Hemanth	Transaction Status Queries
Harshith	View Transaction History
Kartik	Settlement Reports

---

6. Phase-Wise Project Execution Plan

Phase 1: Analysis and Design

- Finalize problem statement and requirements
- Identify major and minor use cases
- Prepare UML diagrams
- Allocate responsibilities

Phase 2: Backend Implementation

- Spring Boot project setup
- Entity and repository creation
- Service and controller implementation
- Design pattern integration

Phase 3: Frontend Implementation

- React UI development
- Individual UI screens per use case
- API integration

Phase 4: Integration and Testing

- Merge all modules into a single application
- End-to-end testing
- Bug fixes
- Screenshot preparation for report

---

7. Compliance with Evaluation Guidelines

- MVC Architecture Pattern: Implemented

- 4 Design Patterns: Implemented
- 4 Design Principles: Applied
- Equal participation ensured
- Each member owns a complete use case
- Single integrated application
- Database persistence enabled